

Generative AI Project

CoverComposer: AI-Powered Music Track Generator

Project Description:

CoverComposer is a Generative AI-powered music generation and management platform that enables users to create instrumental music tracks based on mood, genre, tempo, and style preferences. The platform combines Artificial Intelligence concepts with music composition techniques to generate unique melodies automatically.

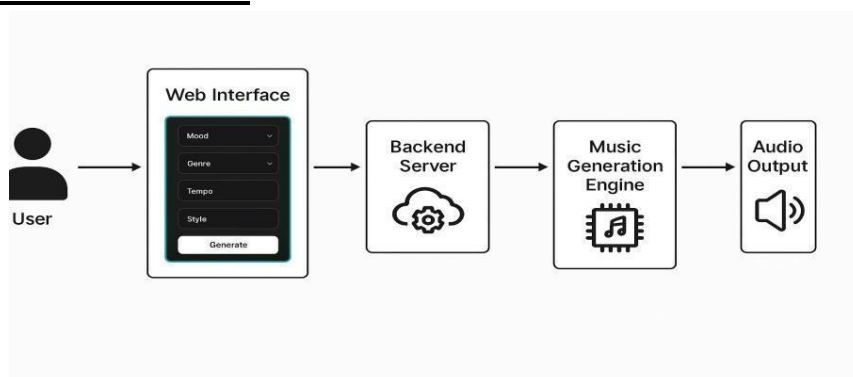
Apart from AI-generated music, the application also provides MP3 song playback, liked songs management, recent song history, user authentication, profile management, and customizable settings. The project aims to deliver a complete music creation and listening experience through a modern and user-friendly web interface.

Scenario:

Scenario 1: Music lovers often want personalized music that matches their emotions and preferences. Traditional music applications only allow users to listen to existing songs and do not provide the ability to generate new music instantly.

Scenario 2: CoverComposer solves this problem by allowing users to generate music dynamically based on selected mood, genre, tempo, and style. Users can also maintain their own music library through liked songs, and recent song history.

Technical Architecture:



Pre-requisites:

1. Software Requirements

- a. **Python 3.x** – Used for backend development and AI music generation.
- b. **Visual Studio Code** – Used for coding and project development.
- c. **SQLite Database** – Used for storing application data.
- d. **FastAPI** – Used to build the web application backend.
- e. **Uvicorn Server** – Used to run the FastAPI application.

2. Libraries Required

- a. **fastapi** – Handles web requests and APIs.
- b. **SQLAlchemy** – Manages database operations.
- c. **Jinja2** – Renders dynamic HTML pages.
- d. **Python-Multipart** – Processes form submissions.
- e. **MIDIUtil** – Generates MIDI music files
- f. **MIDI2Audio** – Converts MIDI into audio.
- g. **FluidSynth** – Creates playable WAV audio files.

Project Workflow:

Activity 1: User Authentication and Navigation

- **Activity 1.1:** Develop user registration and login functionality using FastAPI, SQLAlchemy, and SQLite.
- **Activity 1.2:** Implement session-based authentication for secure user access.
- **Activity 1.3:** Design a dashboard with sidebar navigation for accessing all platform features.

Activity 2: AI Music Generation

- **Activity 2.1:** Create a music generation interface that accepts Mood, Genre, Tempo, and Style as user inputs.
- **Activity 2.2:** Implement AI-based melody generation using predefined musical scales and Markov Chain algorithms.
- **Activity 2.3:** Generate MIDI files dynamically with melody, bass, and drum layers based on user selections.

Activity 3: Audio Processing and Playback

- **Activity 3.1:** Convert generated MIDI files into WAV audio using FluidSynth and SoundFont technology.
- **Activity 3.2:** Provide in-browser audio playback for generated music tracks.
- **Activity 3.3:** Enable downloading of generated audio files for offline listening.

Activity 4: Music Library Management

- **Activity 4.1:** Implement Recent Songs functionality to maintain generated music history.
- **Activity 4.2:** Develop Liked Songs functionality to allow users to save favorite tracks.
- **Activity 4.3:** Provide delete and download options for managing saved songs.

Activity 5: User Personalization and Additional Features

- **Activity 5.1:** Develop user profile functionality displaying account information.
- **Activity 5.2:** Implement playlist management for organizing favorite songs.
- **Activity 5.3:** Create settings functionality for storing user preferences and improving personalization.

Activity 6: MP3 Integration and System Enhancement

- **Activity 6.1:** Integrate predefined MP3 songs alongside AI-generated music.
- **Activity 6.2:** Enable playback, liking, and downloading of MP3 songs.
- **Activity 6.3:** Design a responsive and professional user interface using HTML, CSS, and Jinja2 templates.

Milestone 1: Model Architecture

Name	Date modified	Type	Size
 __pycache__	10-06-2026 18:21	File folder	
 static	09-06-2026 15:02	File folder	
 templates	10-06-2026 14:33	File folder	
 auth	09-06-2026 23:05	Python Source File	0 KB
 covercomposer	10-06-2026 18:10	Data Base File	36 KB
 database	09-06-2026 23:05	Python Source File	1 KB
 main	10-06-2026 18:21	Python Source File	17 KB
 models	10-06-2026 10:26	Python Source File	2 KB
 requirements	09-06-2026 23:05	Text Document	1 KB
 soundfont.sf2	08-06-2026 15:50	SF2 File	1,44,921 KB
 test	09-06-2026 11:22	WAV File	3,259 KB

1. [main.py:](#)

The main.py file acts as the central controller of the CoverComposer application. It manages user authentication, music generation, MP3 playback, song management, playlist handling, profile settings, and all application routes. It also integrates the AI music generation engine, database operations, and frontend templates to provide a complete user experience.

2. [requirements.txt:](#)

The file contains all the Python libraries and dependencies required to run the CoverComposer application. It ensures that the project environment can be recreated easily by installing the necessary packages using a single command.

Key dependencies include FastAPI, SQLAlchemy, Jinja2, MIDIUtil, MIDI2Audio, Uvicorn, and other supporting libraries used for web development, database management, and AI-based music generation.

3. soundfont.sf2:

The soundfont.sf2 file contains instrument sound samples used by FluidSynth to convert MIDI files into realistic audio output. It helps generate high-quality WAV music files with different musical instruments and tones.

4. static/ Folder:

Name	Date modified	Type	Size
output	10-06-2026 18:15	File folder	
songs	10-06-2026 13:16	File folder	
style	10-06-2026 14:17	CSS Source File	7 KB

Serves all static assets used by the application. Includes:

- **style.css**

Aesthetic and professional styling for the CoverComposer application. Contains dashboard layout, sidebar design, responsive UI, buttons, forms, and animations.

- **songs/**

Stores preloaded MP3 songs available for playback within the application. Used by the MP3 Songs module.

- **output/**

Runtime-generated folder that stores:

- .mid files (AI-generated MIDI melodies).
- .wav files (final generated audio tracks).

5. templates/ Folder:

Name	Date modified	Type	Size
 change_password	10-06-2026 14:22	Chrome HTML Do...	1 KB
 dashboard	10-06-2026 14:36	Chrome HTML Do...	2 KB
 home	09-06-2026 23:04	Chrome HTML Do...	1 KB
 index	09-06-2026 23:04	Chrome HTML Do...	2 KB
 liked	10-06-2026 10:53	Chrome HTML Do...	2 KB
 login	09-06-2026 23:04	Chrome HTML Do...	1 KB
 mp3_result	10-06-2026 10:01	Chrome HTML Do...	2 KB
 mp3songs	09-06-2026 23:04	Chrome HTML Do...	2 KB
 profile	09-06-2026 23:04	Chrome HTML Do...	1 KB
 recent	09-06-2026 23:04	Chrome HTML Do...	1 KB
 result	10-06-2026 10:01	Chrome HTML Do...	2 KB
 search	10-06-2026 09:44	Chrome HTML Do...	1 KB
 search_result	10-06-2026 09:43	Chrome HTML Do...	1 KB
 settings	10-06-2026 14:17	Chrome HTML Do...	2 KB
 signup	09-06-2026 23:04	Chrome HTML Do...	2 KB

Contains all HTML pages used in the CoverComposer application.

- **home.html:** Landing page that introduces the CoverComposer platform.
- **signup.html:** Allows new users to create an account.
- **login.html:** Enables registered users to log into the system.
- **dashboard.html:** Main user dashboard containing sidebar navigation and application features.
- **index.html:** AI music generation page where users select mood, genre, tempo, and style.
- **result.html:** Displays the generated music with playback and download options.
- **mp3songs.html:** Allows users to browse and play preloaded MP3 songs.
- **mp3_result.html:** Displays the selected MP3 song for playback and interaction.
- **liked.html:** Shows all songs liked by the user.
- **recent.html:** Displays recently generated or accessed songs.
- **playlists.html:** Manages and displays user playlists.
- **profile.html:** Displays user account information and profile details.
- **settings.html:** Allows users to manage application preferences and settings.
- **search.html:** Provides song search functionality within the application.

Milestone 2 : Backend Development

Import required modules

```
1 from fastapi import FastAPI, Request, Form
2 from fastapi.responses import HTMLResponse, FileResponse
3 from fastapi.staticfiles import StaticFiles
4 from fastapi.templating import Jinja2Templates
5 import subprocess
6 from database import engine, SessionLocal
7 from models import Base, User, LikedSong
8 from fastapi.responses import RedirectResponse
9 from starlette.middleware.sessions import SessionMiddleware
10
11 from midiutil import MIDIFile
12 from midi2audio import FluidSynth
13
14 import random
15 import os
16 from datetime import datetime
```

- **Backend Framework:** FastAPI is used to develop the application backend, handle routing, process user inputs, and manage HTTP requests and responses.
- **Frontend Rendering:** Jinja2 Templates and StaticFiles are utilized to render dynamic web pages and serve static resources such as CSS and audio files.
- **Data Storage:** SQLite database together with SQLAlchemy ORM is used to manage user information, playlists, liked songs, settings, and music records.
- **Music Generation Engine:** MIDIUtil is responsible for creating MIDI compositions based on user-selected parameters, while FluidSynth transforms MIDI data into high-quality WAV audio.
- **Authentication System:** SessionMiddleware enables secure user authentication and maintains active login sessions throughout application usage.
- **Supporting Modules:** Libraries such as os, random, datetime, and subprocess assist with file management, melody randomization, timestamp generation, and external audio processing task.

Initialize FastAPI and Paths:

```
app = FastAPI()
app.add_middleware(
    SessionMiddleware,
    secret_key="covercomposer_secret"
)
Base.metadata.create_all(bind=engine)
```

FastAPI Initialization: `app = FastAPI()` is used to create and initialize the main CoverComposer application.

Session Management: `SessionMiddleware` is added to maintain user login sessions and provide secure access to application features.

Database Initialization: `Base.metadata.create_all(bind=engine)` automatically creates the required database tables if they do not already exist.

Setup static and template Handling

```
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")

templates = Jinja2Templates(directory=TEMPLATE_DIR)

templates = Jinja2Templates(directory=TEMPLATE_DIR)
```

Static File Configuration: `app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")` is used to serve static resources such as CSS files, audio files, and other assets required by the application.

Template Configuration: `Jinja2Templates(directory=TEMPLATE_DIR)` is used to configure the templates folder and render dynamic HTML pages with data from the backend.

Define SoundFont path

```
SOUNDFONT = os.path.join(BASE_DIR, "soundfont.sf2")
```

SoundFont Configuration: `SOUNDFONT = os.path.join(BASE_DIR, "soundfont.sf2")` is used to define the path of the SoundFont file required by FluidSynth for converting MIDI files into realistic WAV audio output.

Define Instrument Presets:

```
GENRE_INSTRUMENTS = {  
    "Pop": 0,  
    "Rock": 29,  
    "Jazz": 26,  
    "Electronic": 81  
}
```

Genre Instrument Mapping: GENRE_INSTRUMENTS is used to assign specific MIDI instrument codes to different music genres, allowing the generated music to match the selected genre style such as Pop, Rock, Jazz, or Electronic.

Generate Melody using Markov Chain

```
def markov_melody(scale, length=32):  
    melody = [random.choice(scale)]  
    for _ in range(length - 1):  
        current = melody[-1]  
        idx = scale.index(current)  
        choices = []  
        if idx > 0:  
            choices.append(scale[idx - 1])  
        choices.append(scale[idx])  
        if idx < len(scale) - 1:  
            choices.append(scale[idx + 1])  
        melody.append(random.choice(choices))  
    return melody
```

Markov Melody Generation: The markov_melody() function generates a sequence of musical notes using a Markov Chain approach. It selects the next note based on the current note and its neighboring notes within the chosen musical scale, creating smooth and natural-sounding melodies.

Stylize Notes with Rhythm & Dynamics

```
def note_properties(style):  
    if style == "Simple":  
        return 1, 100  
  
    duration = random.choice([0.5, 1, 1.5])  
    velocity = random.randint(70, 120)  
  
    return duration, velocity
```

Note Property Generation: The `note_properties()` function determines the duration and velocity of musical notes based on the selected style. Simple mode produces uniform notes, while Complex mode introduces random variations to create more dynamic and expressive music.

Generate Notes Based on Mood

```
MOOD_SCALES = {  
    "Happy": [60, 62, 64, 65, 67, 69, 71],  
    "Sad": [60, 62, 63, 65, 67, 68, 70],  
    "Calm": [60, 62, 64, 67, 69],  
    "Energetic": [60, 64, 67, 72]  
}
```

Mood Scale Mapping: The `MOOD_SCALES` dictionary defines different musical scales for each mood such as Happy, Sad, Calm, and Energetic. These scales are used by the music generation engine to produce melodies that reflect.

Create a Multi-Track MIDI File

```
def create_midi(notes, genre, tempo, style, midi_path):  
    midi = MIDIFile(3)  
  
    melody_track = 0  
    bass_track = 1  
    drum_track = 2  
  
    instrument = GENRE_INSTRUMENTS.get(genre, 0)  
  
    midi.addTempo(melody_track, 0, tempo)  
    midi.addTempo(bass_track, 0, tempo)  
    midi.addTempo(drum_track, 0, tempo)  
  
    midi.addProgramChange(melody_track, 0, 0, instrument)  
    midi.addProgramChange(bass_track, 1, 0, 33)  
  
    time = 0
```

Multi-Layer Music Generation: The `create_midi()` function initializes separate melody, bass, and drum tracks, assigns genre-specific instruments, and sets the tempo for each track. This helps create richer and more realistic AI-generated music.

Convert MIDI to WAV

```
def convert_to_wav(midi_path, wav_path):
```

Audio Conversion: The `convert_to_wav()` function converts the generated MIDI file into a WAV audio file using FluidSynth and the configured SoundFont.

Homepage Route

```
@app.get("/", response_class=HTMLResponse)  
async def home(request: Request):  
  
    return templates.TemplateResponse(  
        request=request,  
        name="home.html"  
    )
```

Home Page Route: The `home()` function handles requests to the application's root URL (`/`) and renders the `home.html` template, serving as the landing page of the CoverComposer platform.

Track Generation Logic

```
@app.post("/generate")
async def generate(
    request: Request,
    mood: str = Form(...),
    genre: str = Form(...),
    tempo: int = Form(...),
    style: str = Form(...)
):

    scale = MOOD_SCALES[mood]

    notes = markov_melody(scale)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    midi_file = os.path.join(
        OUTPUT_DIR,
        f"{timestamp}.mid"
    )
```

Music Generation Process: The `generate()` function receives user-selected mood, genre, tempo, and style inputs, generates a melody using the corresponding mood scale, creates unique timestamp-based MIDI and WAV filenames, and prepares the music generation workflow.

Milestone 3 : Deployment

In Milestone 5, the focus is on deploying the CoverComposer application on a local system. This involves configuring the server environment, ensuring all dependencies are installed and launching the app locally to simulate a real-world setup. This helps verify that the app can run smoothly in a stable environment, allowing you to fine-tune any functionality before deploying it to a cloud platform in the future.

```
cd (folder_location)
```

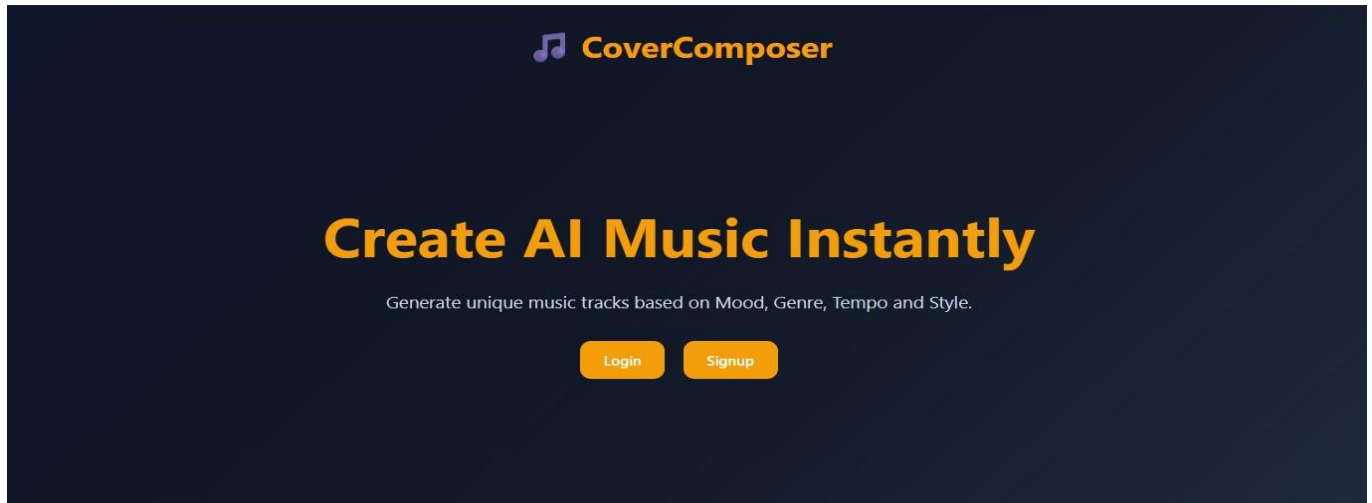
```
pip install -r requirements.txt
```

```
uvicorn main:app --reload
```

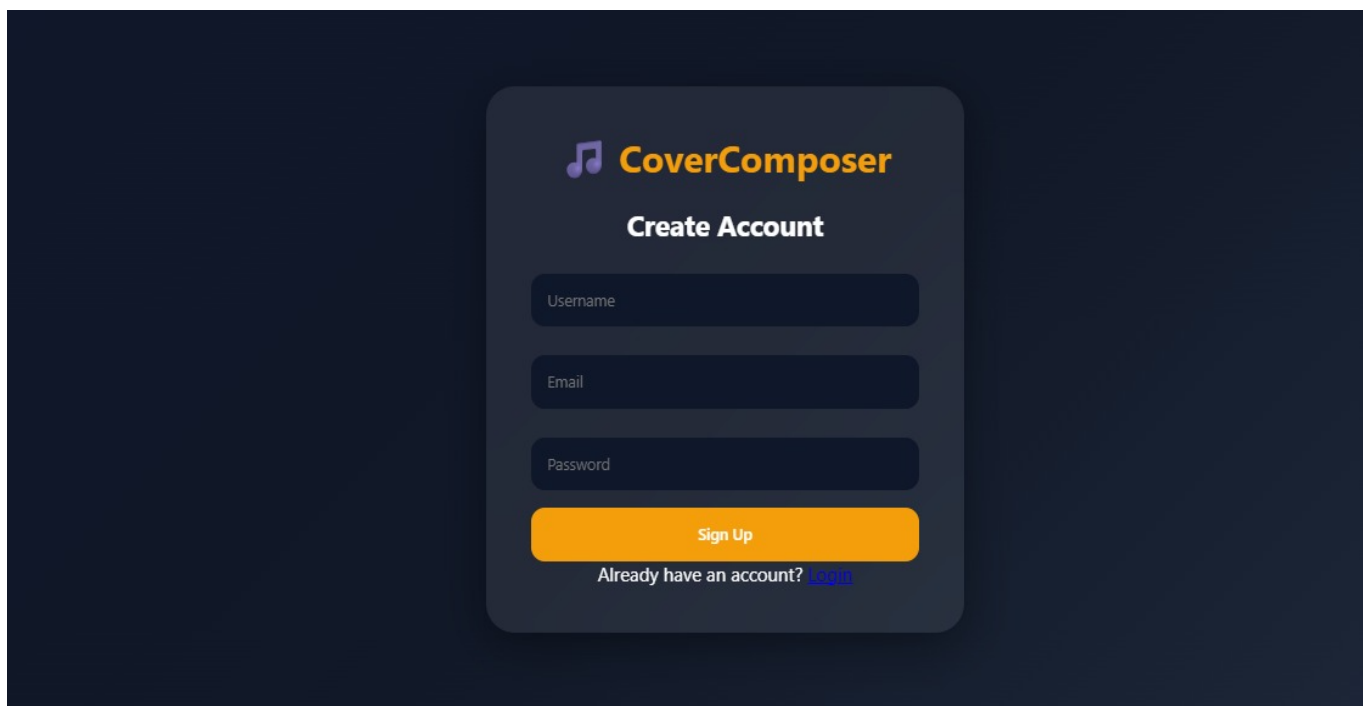
This installed the prerequisite packages and launches the application on **http://127.0.0.1:8000**

Exploring website's Web Pages:

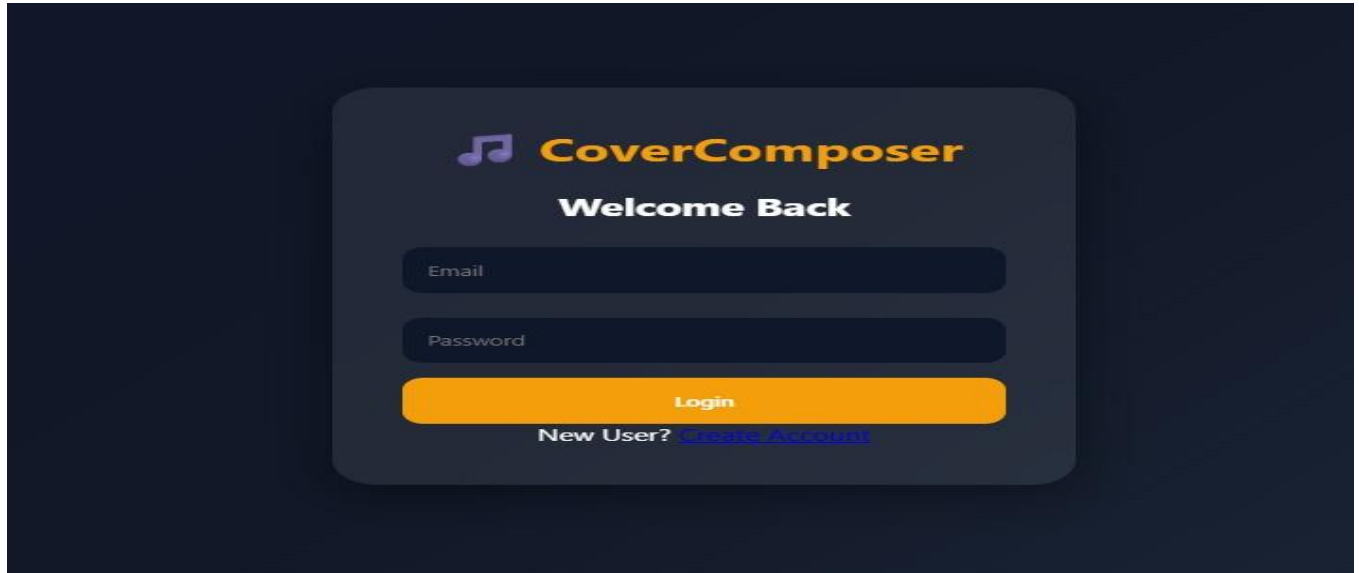
Home Page: This is the Home Page of CoverComposer. It acts as the landing page of the application. Here users can understand the purpose of the platform, which is AI-powered music generation. The page provides navigation options such as Sign Up and Login. The design is simple, attractive, and user-friendly to give a good first impression.



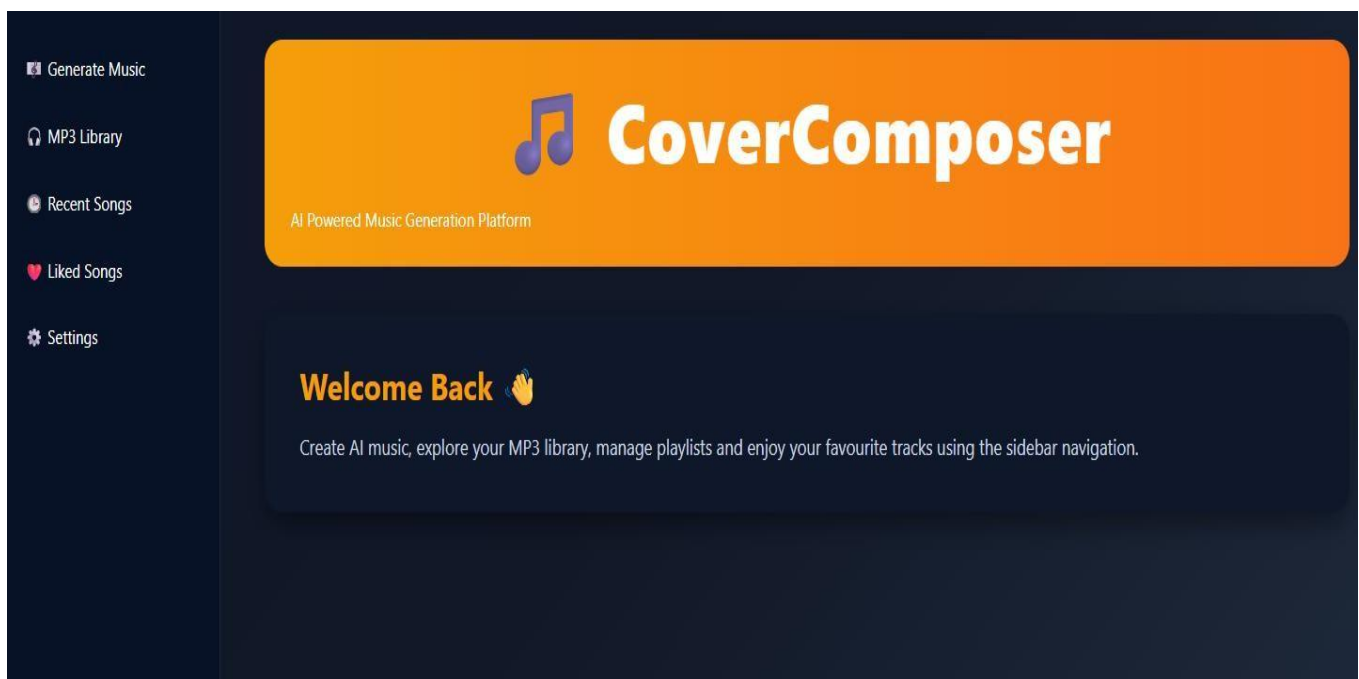
Sign Up Page: This is the Sign Up page. New users can create an account by entering their username, email address, and password. The data is stored securely in the SQLite database through SQLAlchemy. After successful registration, users can access all features of the platform.



Login Page: This is the Login page. Registered users can enter their credentials to access the system. FastAPI verifies the username and password and creates a secure session using SessionMiddleware. After successful login, the user is redirected to the dashboard.



DashBoard: This is the main Dashboard of CoverComposer. It acts as the control center of the application. The sidebar provides quick navigation to AI Music Generator, MP3 Library, Recent Songs, Liked Songs, Playlists, Profile, and Settings. From here users can access every feature available in the platform.



AI Music Generator: This page is the core feature of CoverComposer. Users can generate music by selecting Mood, Genre, Tempo, and Style. Based on these inputs, the AI music generation engine creates a unique melody using Markov Chain algorithms and predefined musical scales.

Genre Pop:



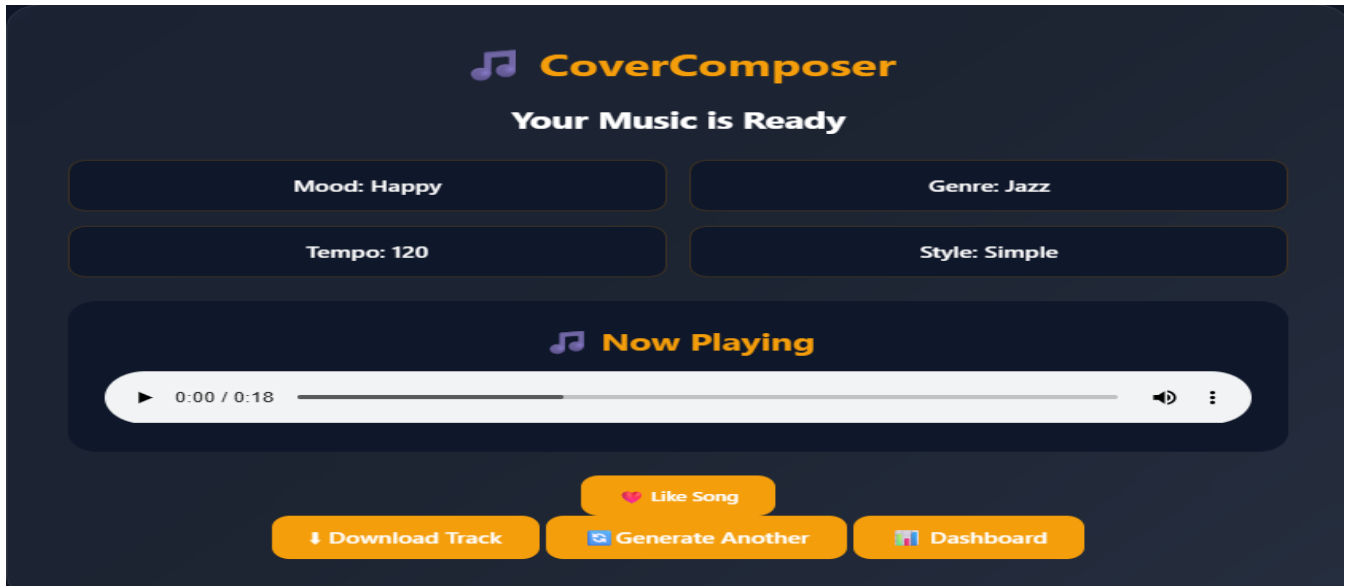
The interface for the CoverComposer AI Music Generator is displayed. It features a dark blue background with a title bar at the top that reads "CoverComposer AI Music Generator" in orange and white text. Below the title bar, there are four input fields for selecting music parameters: "Mood:" with a dropdown menu showing "Happy", "Genre:" with a dropdown menu showing "Pop", "Tempo:" with a text input field showing "120", and "Style:" with a dropdown menu showing "Simple". Each dropdown menu has a small downward arrow icon. At the bottom center, there is a yellow button labeled "Generate Music".

Genre Rock:



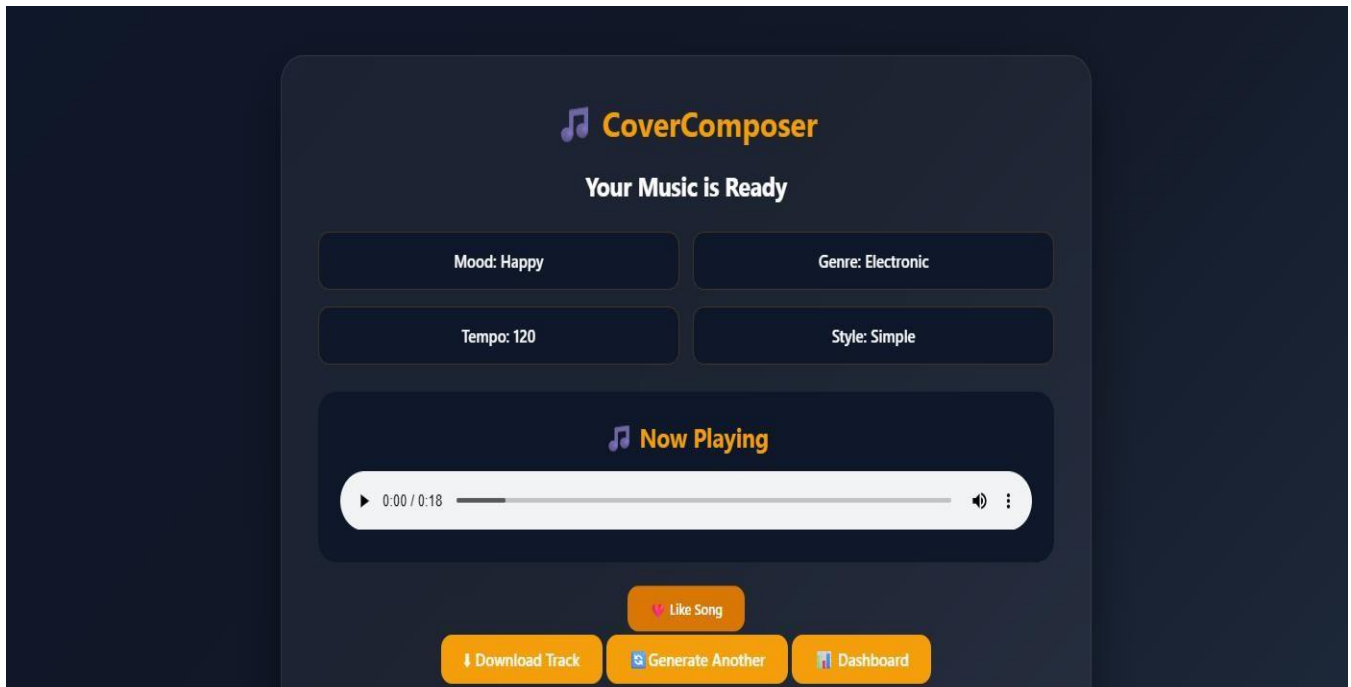
The interface for the CoverComposer AI Music Generator is displayed, showing the results of a music generation process. The title bar at the top reads "CoverComposer" in orange and white text, followed by the subtitle "Your Music is Ready" in white text. Below the title bar, there are four input fields for selecting music parameters: "Mood: Happy", "Genre: Rock", "Tempo: 120", and "Style: Simple". Each input field is a dark blue button with white text. Below these input fields, there is a section titled "Now Playing" in orange and white text. This section contains a progress bar with a play button icon on the left, a time display showing "0:00 / 0:18", and a volume icon on the right. At the bottom, there are three yellow buttons: "Download Track", "Like Song" (with a heart icon), and "Generate Another" (with a refresh icon). A "Dashboard" button with a home icon is also visible at the bottom right.

Genre Jazz:



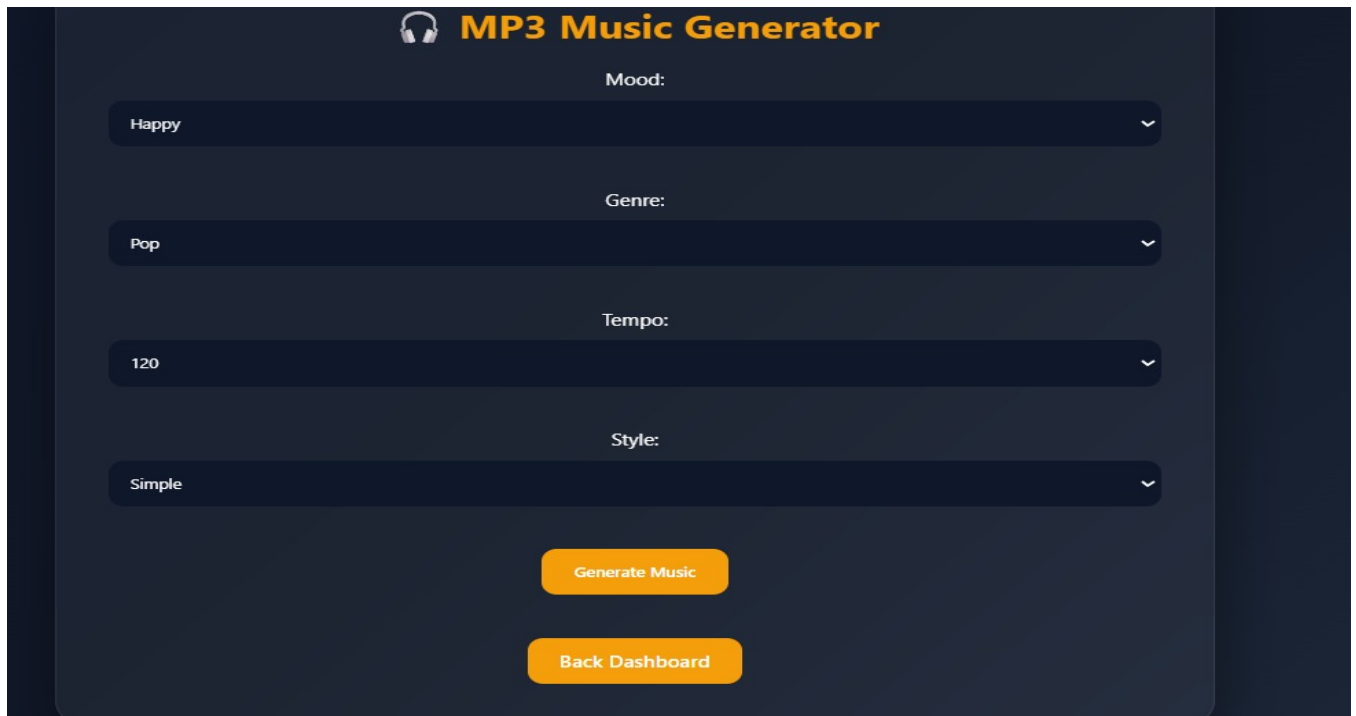
The screenshot shows the CoverComposer web application interface. At the top, the title "CoverComposer" is displayed with a musical note icon, followed by the subtitle "Your Music is Ready". Below this, there are four buttons arranged in a 2x2 grid: "Mood: Happy", "Genre: Jazz", "Tempo: 120", and "Style: Simple". Underneath these buttons is a "Now Playing" section with a progress bar showing "0:00 / 0:18" and a play button. At the bottom, there are three buttons: "Download Track", "Like Song", and "Generate Another", along with a "Dashboard" button.

Genre Electronic:



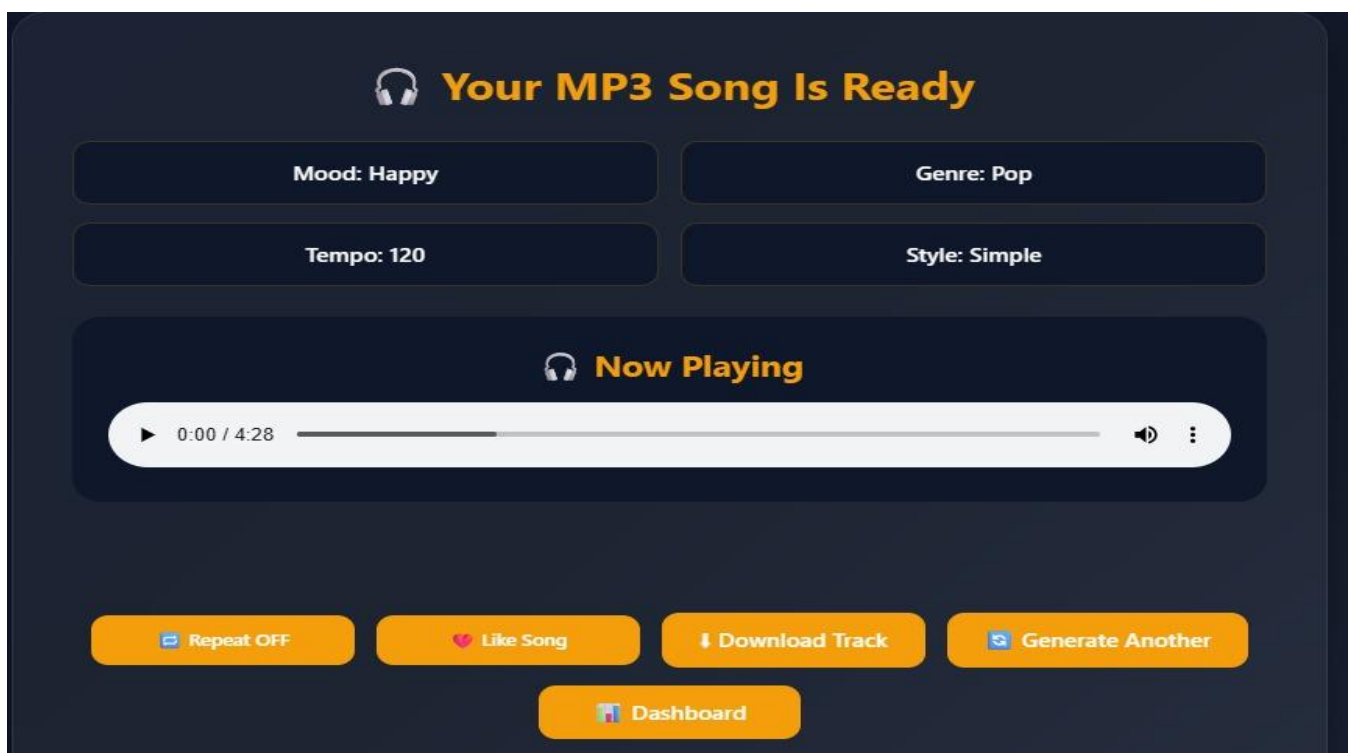
The screenshot shows the CoverComposer web application interface with the genre set to "Electronic". The layout is identical to the previous screenshot, but the "Genre: Jazz" button has been replaced by "Genre: Electronic". The "Now Playing" section and the bottom buttons remain the same.

MP3 Music Library: This is the MP3 Music Library page. Apart from AI-generated music, CoverComposer also provides predefined MP3 songs. Users can browse, play, like, and download these songs."



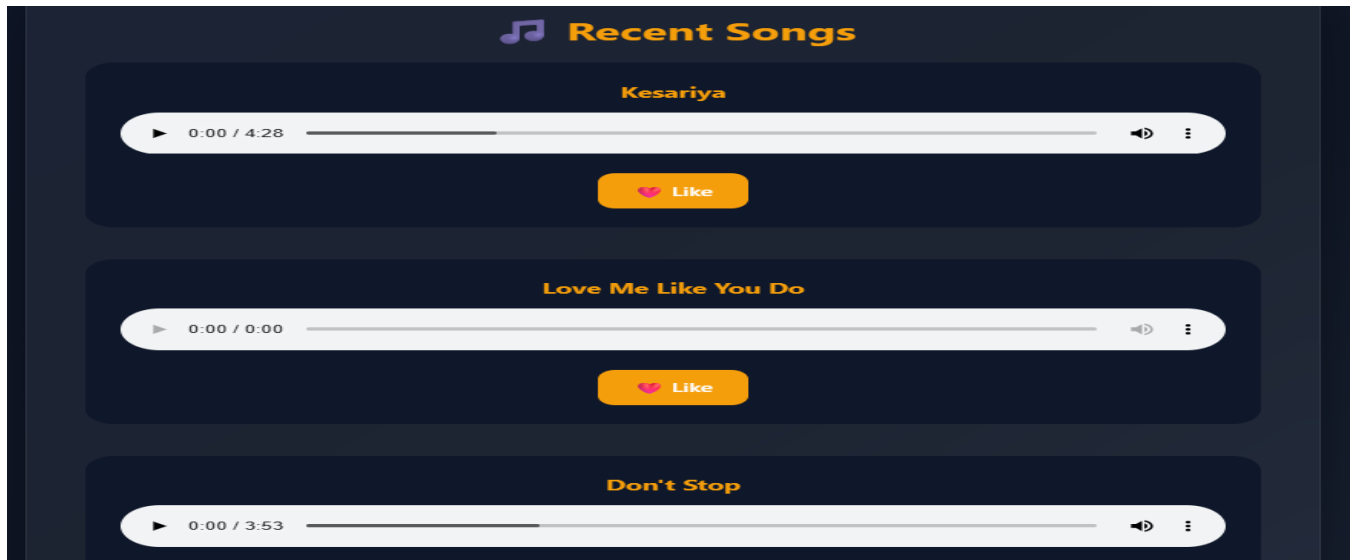
The interface is titled "MP3 Music Generator" with a headphones icon. It features four dropdown menus for selecting song parameters: "Mood" (set to "Happy"), "Genre" (set to "Pop"), "Tempo" (set to "120"), and "Style" (set to "Simple"). Below these menus are two orange buttons: "Generate Music" and "Back Dashboard".

After Generation:

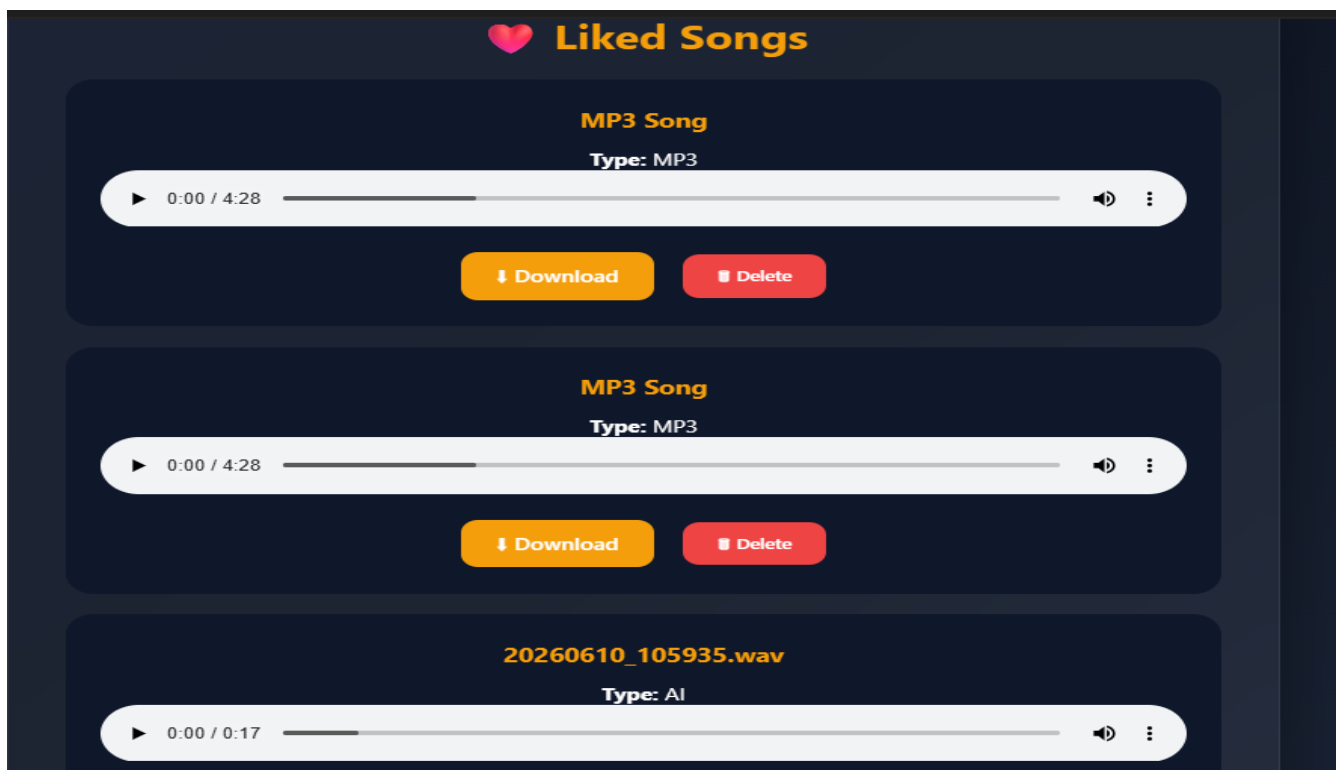


The interface is titled "Your MP3 Song Is Ready" with a headphones icon. It displays the generated song's parameters in four boxes: "Mood: Happy", "Genre: Pop", "Tempo: 120", and "Style: Simple". Below these is a "Now Playing" section with a progress bar showing "0:00 / 4:28" and a volume icon. At the bottom are five orange buttons: "Repeat OFF", "Like Song", "Download Track", "Generate Another", and "Dashboard".

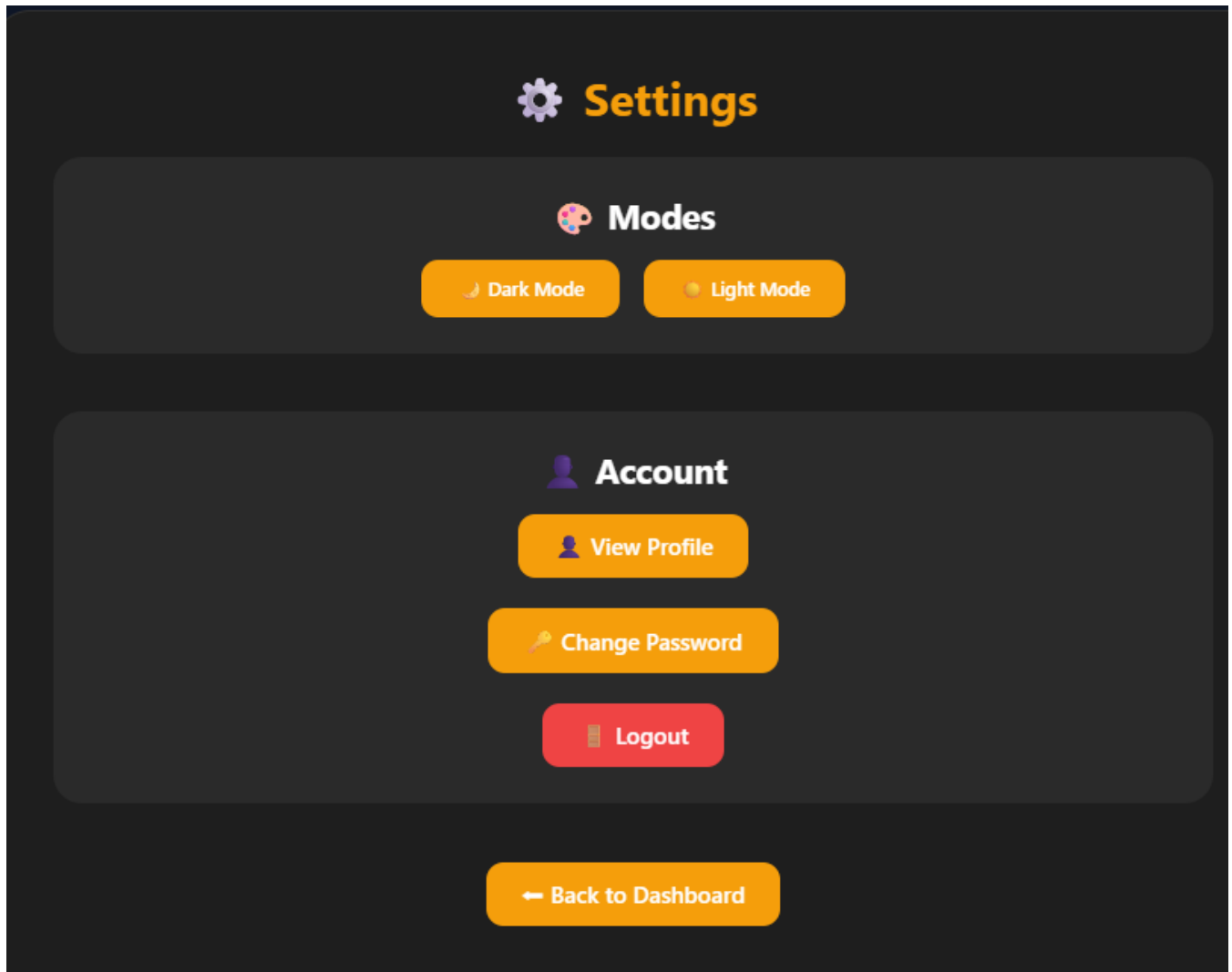
Recent Songs: This page displays recently generated or accessed songs. It helps users quickly revisit their previous music creations without generating them again.



Liked Songs: This is the Liked Songs page. Users can save their favorite AI-generated or MP3 tracks for quick access in the future.



Settings: This is the Settings page. Users can customize application preferences and personalize their experience. Future versions can include theme settings, audio quality settings, notification preferences, and account management options.



FUTURE ENHANCEMENTS:

- AI Lyrics Generation
- Voice-Based Music Creation
- Music Recommendation System
- Cloud Storage Integration
- Playlist Sharing
- Mobile Application Development
- Real-Time Music Collaboration
- Deep Learning-Based Music Generation Models

CONCLUSION:

CoverComposer successfully demonstrates the practical application of Generative AI in music creation and management. The system enables users to generate unique music tracks, manage personal music collections, maintain playlists, and enjoy a personalized listening experience. By integrating AI technologies with modern web development frameworks, CoverComposer provides a comprehensive platform for intelligent music generation and management.